

COS 214 FINAL PROJECT

Momina and Friends ✨

Mahadio Tlaka u23545098, Diya Narotam u23533596, Chinmayi Santhosh u24585671, Shavir Vallabh u23718146, Jaitin Moodally u23621372, Ayush Beekum u23596351, Fabio Berrino u23526387

Google Docs Link:

<https://docs.google.com/document/d/1wlvLGTEzyj20h6J57he9fmpKkfqXPxbo/edit?usp=sharing&oui=113333442838782430246&rtpof=true&sd=true>

Research Overview:

The research for this project focused on understanding how real-world nurseries operate, the factors affecting plant growth, and the management systems used to monitor and maintain plant health. Key research areas included plant classification, growth requirements, and nursery management practices.

A nursery is a controlled environment where various plants are cultivated for commercial or research purposes. Effective nursery management involves organizing plants based on their characteristics such as size, type, and environmental needs. Each plant category requires specific care routines such as water regulation, temperature control, and exposure to sunlight, to ensure optimal growth.

Four main categories of plants were researched for this project:

Tropical Plants: Require warm temperatures, high humidity, and consistent watering. Examples include *Bird of Paradise* and *Pitcher Plants*.

Carnivorous Plants: Adapted to nutrient-poor soils, these plants rely on trapping insects for nutrients. Examples include *Sundew* and *Nepenthes*.

Succulent Plants: Store water in their leaves and thrive in dry environments. Examples include *Aloe Vera* and *Hen and Chick*.

Temperate Plants: Grow in moderate climates and often have seasonal growth patterns. Examples include *Lilac* and *White Oak*.

This classification directly influenced the GreenHouse design in our simulation, where each plant type is managed under its own factory, ensuring accurate representation of environmental needs.

Influence on Design Decisions:

The design of our GreenHouse simulation is driven using multiple **object-oriented design patterns** that collectively ensure modularity, flexibility, and realism. Each pattern was carefully selected to solve a specific design challenge encountered in simulating a functioning plant nursery. For instance, creational patterns such as **Abstract Factory** and **Prototype** streamline plant creation by ensuring that new plant types can be easily instantiated or cloned without modifying existing code. Meanwhile, **Structural patterns** like **Decorator** and **Composite** enhance scalability by allowing dynamic addition of features (such as pots or fertilizer) and by efficiently organizing customers and company groups within the sales subsystem.

On the behavioural side, patterns such as **State**, **Observer**, **Command**, and **Chain of Responsibility** bring the simulation to life by modelling plant growth, health monitoring, and automated care. The **Iterator** pattern ensures seamless traversal of the plant inventory stored in a BST, while the **Mediator** facilitates communication between staff, plants, and customers, reducing tight coupling. Finally, the **Memento** pattern provides reliability by enabling stock or command recovery through system snapshots. Together, these patterns create a highly interactive and autonomous simulation, mirroring the real-world operations of a nursery while maintaining code reusability, scalability, and ease of maintenance.

Design Pattern Application

This section details the design patterns implemented in the Greenhouse Nursery Simulation, explaining their purpose, implementation, and how they collectively enhance system flexibility, modularity, and maintainability.

Prototype Pattern

Category: Creational

Purpose in Our Design:

Enables cloning of existing Plant objects to efficiently handle bulk orders or duplicate plants with similar attributes without recreating them from scratch.

Implementation Summary:

The Plant base class defines a clone() method that allows derived classes to return copies of themselves.

When multiple plants of the same type are requested, the system replicates prototypes rather than reinitializing each one, optimizing performance. Beyond code optimization, this can be likened to ‘grafting’ a plant for a real world parallel.

```

class Plant {
private:
    mSpecies : string
    mWaterLevel : int
    mSunlightLevel : int
    mFertilizerLevel : int
    mPruneLevel : int
    mGrowthStage : int
    mCareStrategy : PlantCareHandler*
    mGrowthState : GrowthState*
    mHealthState : HealthState*
    mClimate : string
    mDescription : string
    mPrice : double
    mSize : string
    mCurrentCycleCount : int
    mSeedCyclesNeeded : int
    mSproutCyclesNeeded : int
    mMatureCyclesNeeded : int
    mObserver : ConcreteGrowthObserver*
    mReadyForStock : bool
public:
    ~Plant()
    +Plant(species : string)
    +Plant(const other : Plant&)
    +Plant()
    +getPrice() : double
    +getDescription() : string
    +getClimate() : string
    +clone() : Plant*
    +setCareStrategy(strategy : PlantCareHandler*) : void
    +setGrowthState(state : GrowthState*) : void
    +setHealthState(state : HealthState*) : void
    +attach(observer : ConcreteGrowthObserver*) : void
    +detach() : void
    +notify() : void
    +getSpecies() : string
    +setPrice(newPrice : double) : void
    +setDescription(newDesc : string) : void
    +tick() : void
    +isReadyForStock() : bool
    +markReadyForStock() : void
    +getWaterLevel() : int const
    +restoreWater() : void
    +getSunlightLevel() : int const
    +restoreSunlight() : void
    +getFertilizerLevel() : int const
    +restoreFertilizer() : void
    +getPruneLevel() : int const
    +restorePrune() : void
    +completeCareSession() : void
    +getPrice() : string const
    +getCurrentCycleCount() : int const
    +getSeedCyclesNeeded() : int const
    +getSproutCyclesNeeded() : int const
    +getMatureCyclesNeeded() : int const
    +resetCycleCount() : void
    +printCurrentNeeds() : void
}

```

Figure 1: The abstract Plant class, specifying the Clone() method required for Prototype.

Abstract Factory Pattern

Category: Creational

Purpose in Our Design:

Provides an interface for creating related plant objects without specifying their exact classes, ensuring consistent plant creation across various greenhouse environments.

Implementation Summary:

The GreenHouse class acts as the **Abstract Factory**, defining methods for creating plants of different categories. Factories such as TropicalPlantFactory, CarnivorousPlantFactory, SucculentPlantFactory, and TemperatePlantFactory each implement plant creation specific to their climate.

Abstract products (Plant) and concrete subclasses (SmallPlant, MediumPlant, LargePlant) represent plant categories, ensuring consistent characteristics across all product families. This allows each factory to instantiate plants associated with its climate, differentiating by size.

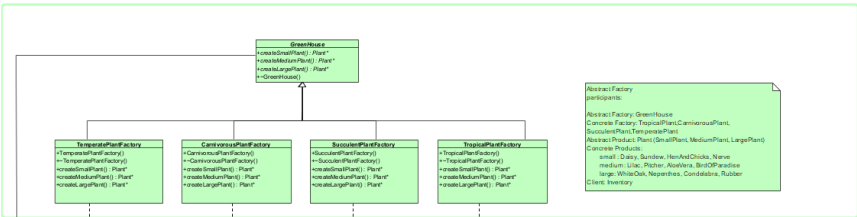


Figure 2: The Abstract Factory hierarchy, specifying different greenhouse conditions.

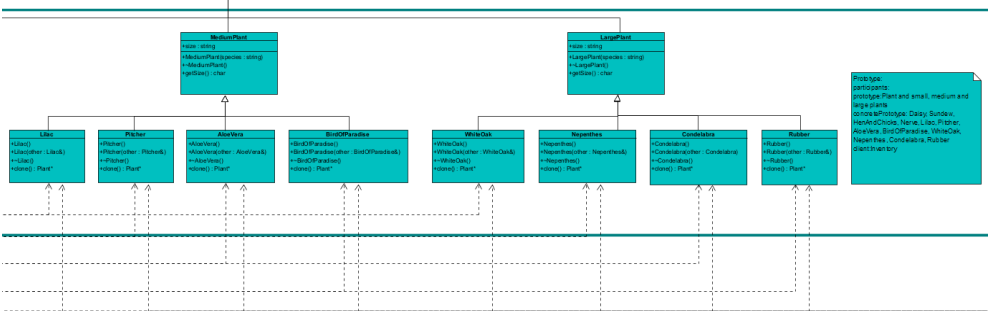


Figure 3: The Product hierarchy of the Abstract Factory pattern. A three-tiered hierarchy for logical groupings, all inheriting from Plant.

Decorator Pattern

Category: Structural

Purpose in Our Design:

Adds extra features and behaviours to Plant objects dynamically without modifying their original class.

Implementation Summary:

PotDecorator and FertilizerDecorator extend the base Plant functionality.

PotDecorator adds structural modifications (e.g., decorative pots), while FertilizerDecorator provides behavioural enhancements that influence plant growth. This ensures flexible feature addition without subclass explosion and neatly allows for plants to be customised by customers whilst maintaining a uniform interface.

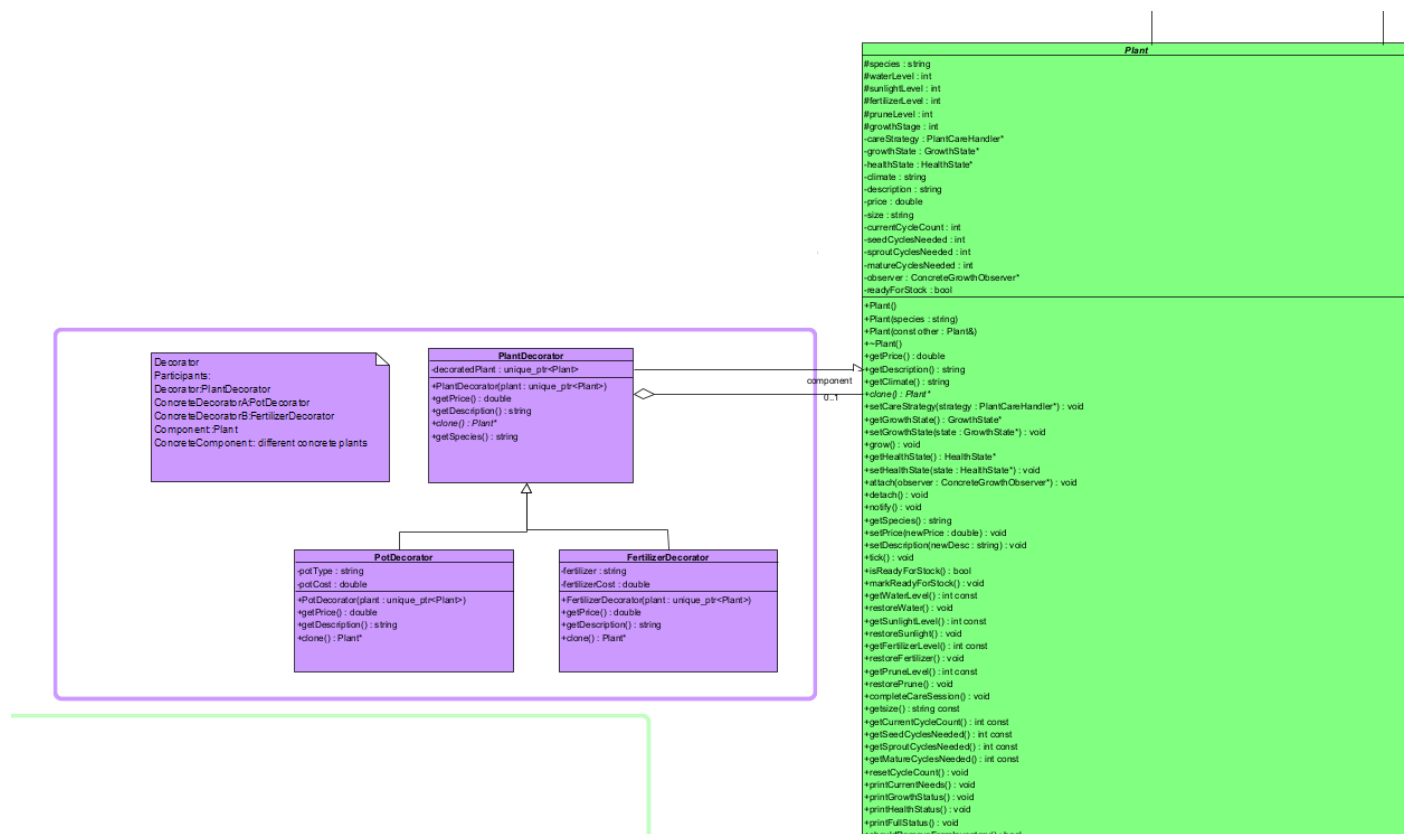


Figure 4: The Decorator pattern attached to Plant, featuring pot and fertilizer decorators

Composite Pattern

Category: Structural

Purpose in Our Design:

Organizes customers into a hierarchy of individuals and company clients, allowing both to be treated uniformly.

Implementation Summary:

Customer is the base component class. A company may consist of multiple different customers, with the individual customer types being 'Normal' and 'Premium'. This allows for easy and logical groupings of clientele, as well as personalized discounts, which can be likened to bulk order discounts and loyalty programmes.

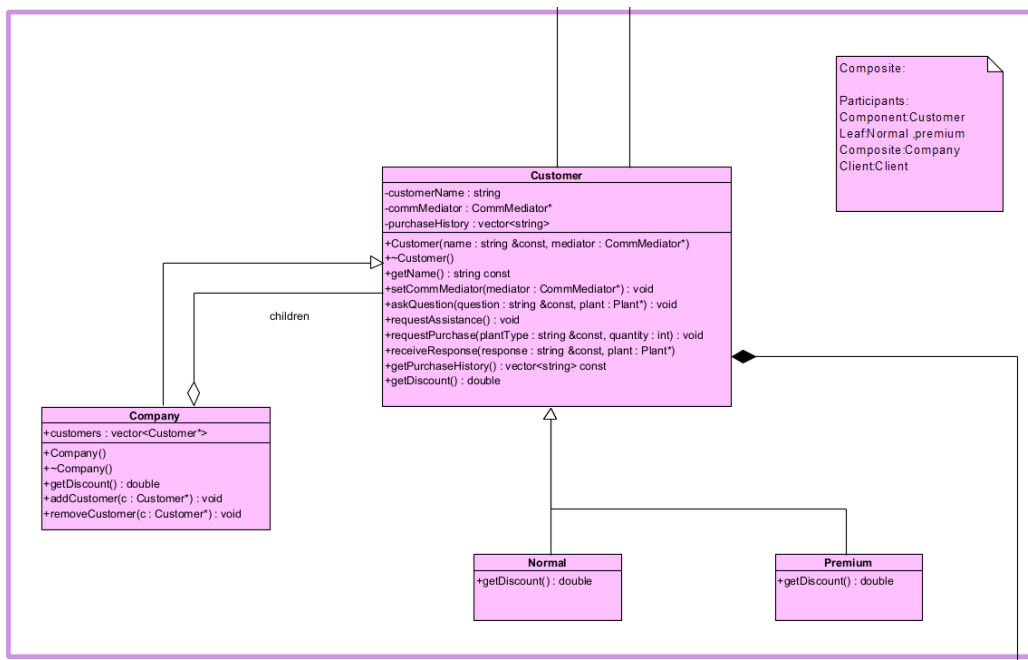


Figure 5: The Customer hierarchy, allowing for customers to be grouped in a tree-like structure

Iterator Pattern

Category: Behavioural

Purpose in Our Design:

Facilitates traversal through the nursery's inventory system without exposing internal data structures.

Implementation Summary:

Inventory and **Stock** are implemented as **Binary Search Trees (BSTs)** where each node represents a plant species, where each node stores a vector of Plant pointers. The Iterator classes provide in-order traversal of BST nodes and their internal vectors, enabling alphabetical iteration of plant species. The iterators for these classes also feature fine-grained and coarse-grained traversals, allowing sequential iteration over each Plant in the structures, or each node in the structure, allowing for faster searching.

Each Plant object has attached to it a GrowthObserver to monitor its health. When a plant's health drops below 60%, the observer alerts a StaffMember, triggering the necessary care actions. Promotes a one-to-many relationship, ensuring scalability across multiple plants.



Command Pattern

Category: Behavioural

Purpose in Our Design:

Encapsulates plant care actions as command objects that staff members execute to restore plant health.

Implementation Summary:

Commands include WaterCommand, SunCommand, FertilizerCommand, and PruneCommand. StaffMember acts as the **Invoker**, executing commands based on observer notifications. Each command modifies plant attributes and can be stored or replayed for consistency. Each of these commands improve their respective statistic for each plant, and contribute towards plant growth.

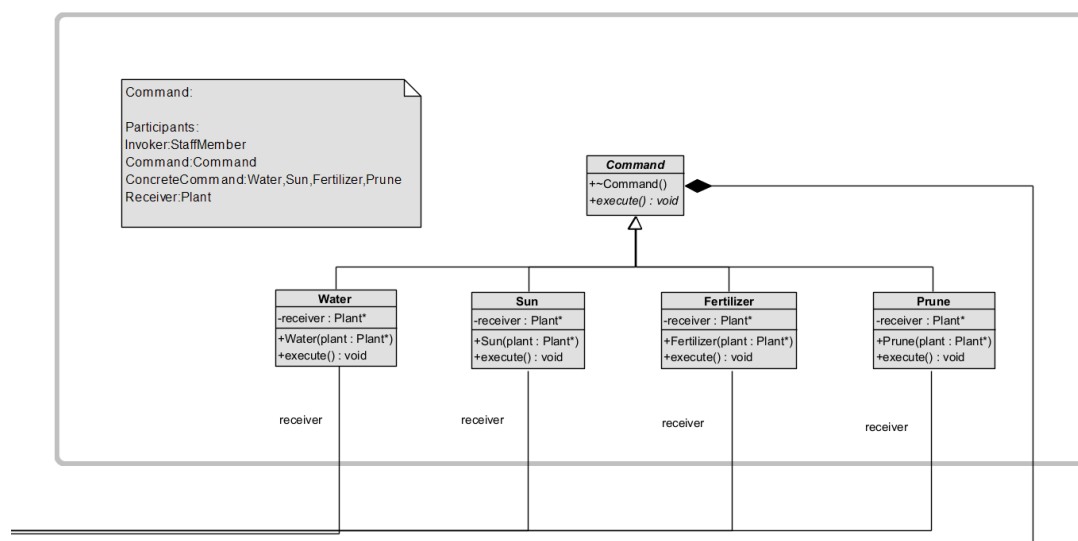


Figure 8: The Command hierarchy, used for plant care.

Chain of Responsibility Pattern

Category: Behavioural

Purpose in Our Design:

Links multiple plant-care handlers to automatically determine and respond to specific plant needs.

Implementation Summary:

The `tick()` function in **Plant** gradually decreases care levels. When levels drop below 60%, the **GrowthObserver** triggers the **Chain of Responsibility**.

This pattern works seamlessly with **Command**, as **Handlers** including **WaterHandler**, **SunlightHandler**, and **FertilizerHandler** check requirements in order and invoke the proper **Command**.

This decouples detection from the action, improving maintainability.

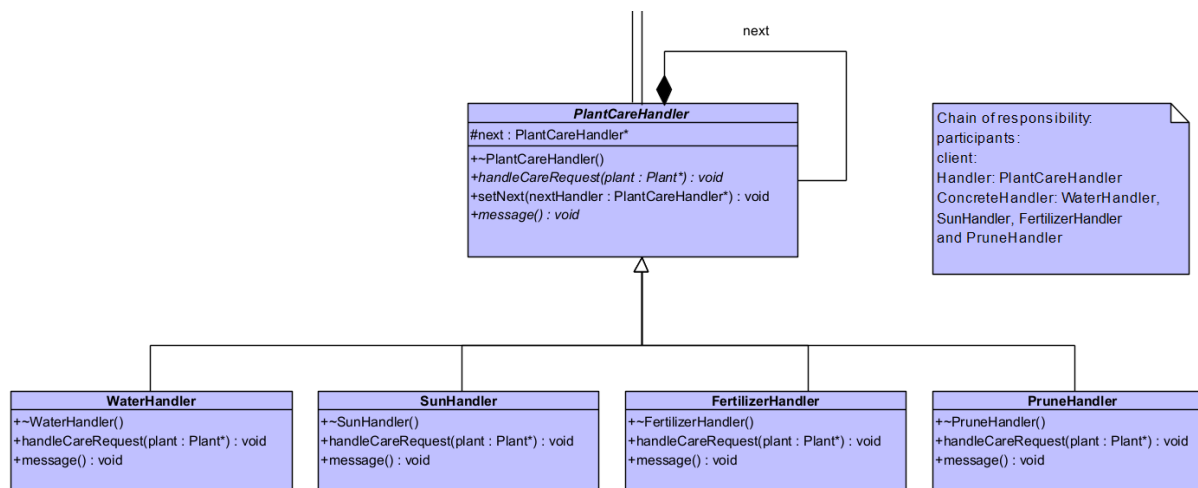


Figure 9: PlantCareHandler hierarchy, each allowing for a specific care action to be handled.

Mediator Pattern

Category: Behavioural

Purpose in Our Design:

Coordinates communication between objects, reducing direct dependencies. The mediator serves two purposes in our design, both involving StaffMembers

Implementation Summary:

GrowthMediator facilitates communication between Plant and StaffMember, whereas CommMediator manages communication between Customer and StaffMember. This greatly reduces communication complexities and overhead, allowing for simple and extensible many-to-many communication

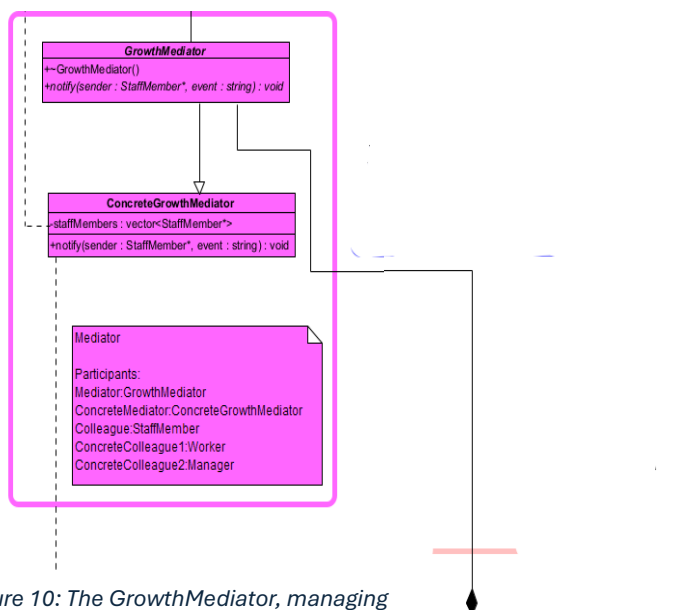


Figure 10: The GrowthMediator, managing communication between Plants and StaffMembers

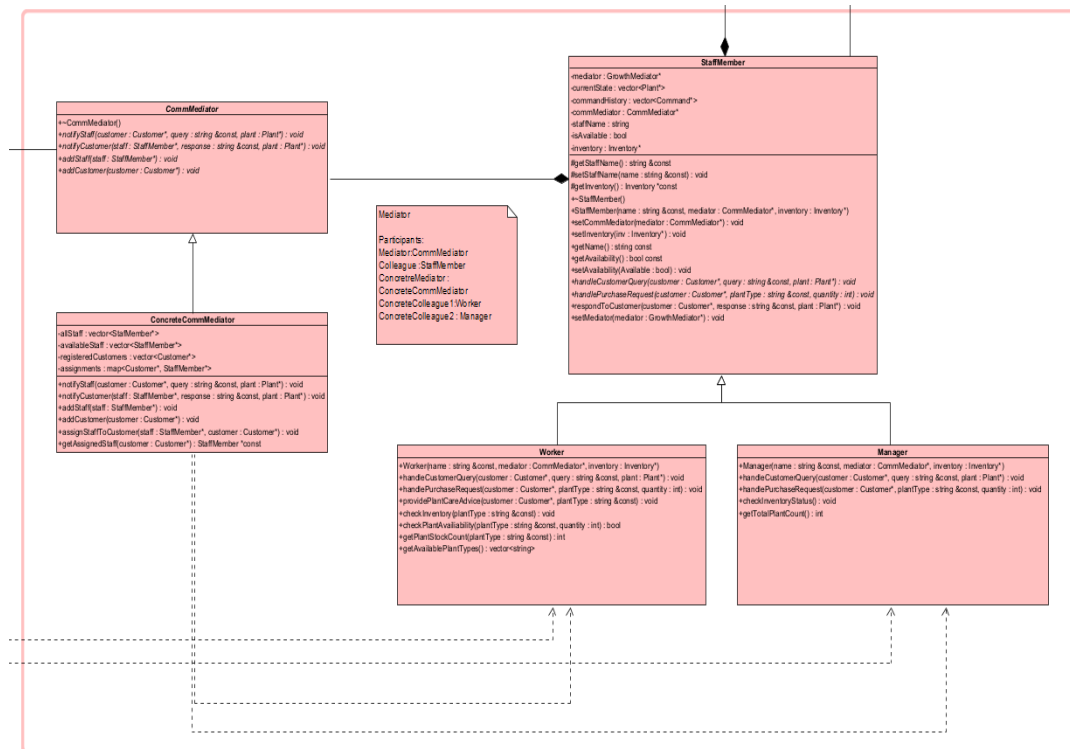


Figure 11: The CommMediator, handling communication between StaffMembers and Customers with queries

Memento Pattern

Category: Behavioural

Purpose in Our Design:

Allows a snapshot of any object's state to be created, stored, and restored in the system. Mainly used for capturing The Stock and storing orders at a certain point in time.

Implementation Summary:

Templatized to allow Mementos to be created for any class, storing a deep copy of its current state. Before large operations or stock updates, a Memento captures the current inventory snapshot, allowing stress-free rollback. This simulates a real database system where failsafes should be put in place, such as transactions, in case of errors.

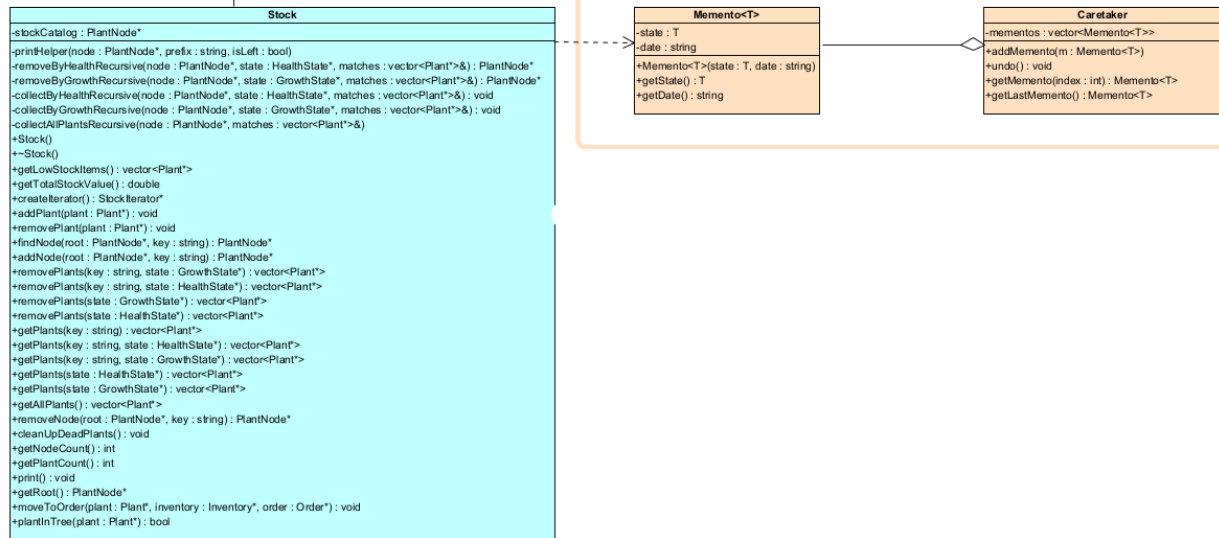


Figure 12: Memento, templatized but pictured here working with Stock

State Pattern

Category: Behavioural

Purpose:

Implemented in two classes, serving different purposes. Allows for Plant's growth and health to be modelled, as well as tracking an order.

Implementation Summary:

GrowthState, HealthState, and OrderState manage lifecycle and transaction flow. These classes simplify complex state transitions by keeping behaviour within individual state classes. They allow for different

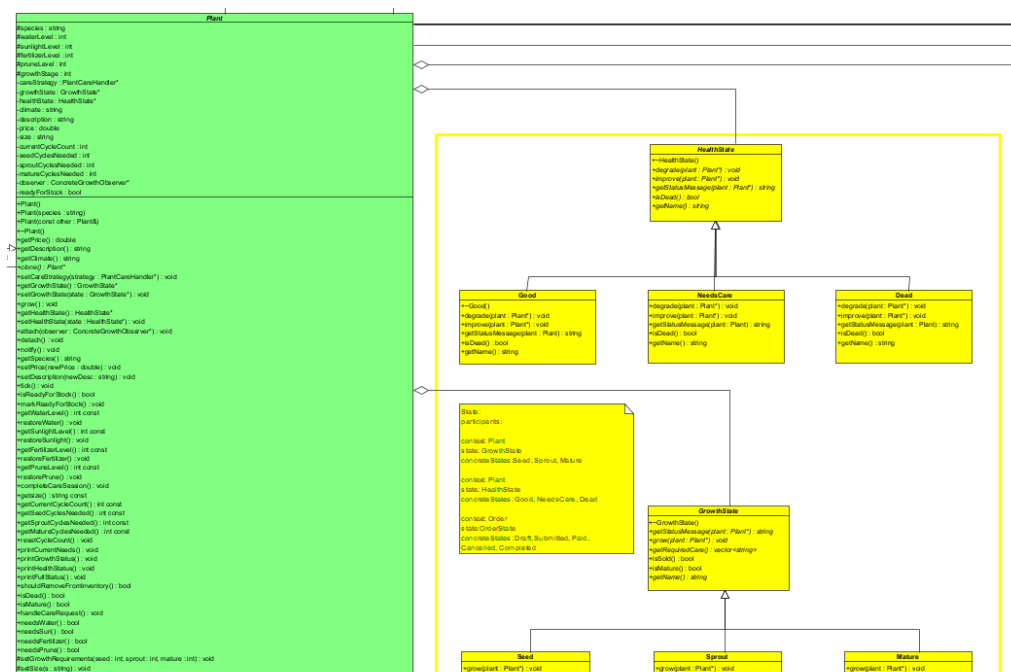


Figure 13: Growth and Health states for Plants, the first instance of State in our project

actions to be possible depending on the state of its related object, and reduce the complexity of their associated classes by encapsulating this behaviour. They are used extensively for automation, such as cleaning up dead plants, requesting for plant care, or moving plants between inventory and stock.

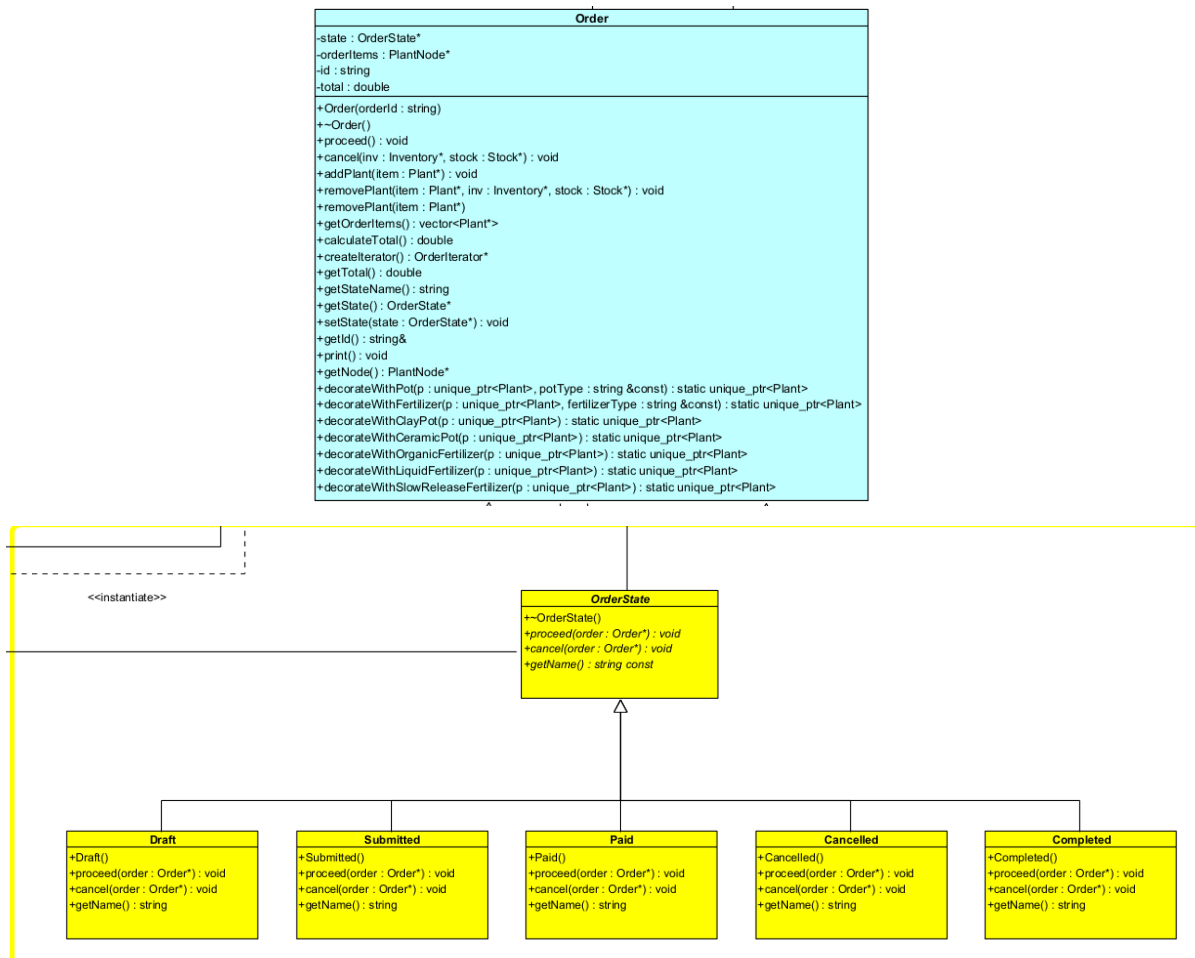


Figure 14: Order and OrderState, the second instance of State.

Conclusion:

The modelling and development of the nursery simulation demonstrated the power of OOP design patterns in software development, creating a flexible, scalable and maintainable system. By combining various structural, creational and behavioural patterns, we successfully modelled a lively and realistic simulation of a nursery, featuring real time plant growth/decay, customer management, and automated plant care.

Each design pattern addressed a specific challenge, spanning from object creation and feature extension to communication and state management. Patterns such as Abstract Factory and Prototype ensured consistent and efficient plant generation, while Decorator and Composite provided extensibility and logical organization. Behavioural patterns like State, Observer, and Command introduced lifelike behaviour and interactivity, turning the simulation into an autonomous ecosystem rather than a static model.

Ultimately, this project illustrates how thoughtful application of design patterns can transform a complex or otherwise convoluted domain into an elegant software architecture that mirrors real-world operations while remaining adaptable for future growth and innovation.